UCS503P – Software Engineering Project

Git-Mind

A Verification-Gated, Human-in-the-Loop CI/CD Assistant

Submitted by:

1024030368	Mohit Srivastav
1024030361	Seerat
1024030384	Shreyas Tiwari
1024030353	Yuvraj Malik

Group: 3C41 BE Third Year, CSE

Submitted to:

Miss Nisha Thakur

Project Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1

Introduction

1.1 Background and Context

Continuous integration (CI) provides rapid feedback on code changes, but a failed test or build still interrupts a developer's work. Even small faults such as a missing import, an off-by-one error, or incorrect error handling require someone to inspect the log, locate the relevant code, prepare a patch, and rerun the test suite. Git-Mind is a web-based assistant intended to reduce this routine toil while retaining human control over every change.

The system combines a React dashboard, an Express API, a Redis-backed worker, MongoDB persistence, and an LLM-driven fixer. A GitHub failure event or a manual trigger creates a repair job. The worker proposes complete replacement content for the affected file, applies it only inside a restricted local repository, executes the corresponding test when available, and creates a reviewable pull request only after successful verification. GitHub webhooks provide the event model used for CI integration [Git26a]; BullMQ supports decoupled, asynchronous processing [Tas26].

1.1.1 Problem Statement

Existing automated repair tools can produce plausible patches without proving that they preserve the expected behaviour. Unverified automation is risky, particularly when it can commit to a shared repository. The project therefore addresses the following problem: *how can routine, deterministic CI failures be converted into candidate fixes without allowing an untested AI output to bypass developer review?*

1.2 Project Objectives

1. Build an event-driven workflow that records a failed CI check and queues a repair task without blocking the API service.

2. Generate candidate code fixes from failure context and file content, with a maximum of three attempts for retryable failures.

3. Verify every candidate against the associated local test before it is committed; restore the original content if verification fails.

4. Deliver verified changes through a separate Git branch and pull request, preserving a mandatory human review and merge decision.

5. Provide an observable workspace that displays repositories, pull requests, activity, error logs, and AI-fix status.

Chapter 2

Methodology and Design

2.1 System Architecture

Git-Mind is organised as a Node.js monorepo with four workspaces: frontend, backend, AI worker, and shared models. The design separates failure ingestion from LLM-bound repair work so that external webhooks receive a prompt response while longer-running jobs are processed independently.

2.1.1 High-Level Design

The frontend presents a single workspace containing a repository activity graph, live metrics, error-log panel, and code-question drawer. It uses React and React Flow for the interactive view [Met26; Rea26]. The backend exposes REST endpoints for repositories, commits, pull requests, activity, AI logs, chat, manual repair triggering, authentication, and GitHub webhooks. Socket.io emits fix-related events to the dashboard.

On a failed `check_run`, the webhook router records the event, marks the related pull request as failed, emits an `AI_FIX_STARTED` notification, and adds a `fix-code` job to BullMQ. The worker invokes the fixer agent. The agent supplies the error context and selected files to a Gemini model through LangChain, extracts fenced full-file code blocks, and passes them to the Git operations module. MongoDB stores repository, pull-request, and AI-log state through Mongoose [Mon26].

2.1.2 Technical Stack

- **Frontend:** React, Vite, React Router, Zustand, React Flow, Axios, and Socket.io Client.
- **Backend:** Node.js, Express, Socket.io, Octokit, JWT, Mongoose, and CORS.
- **Asynchronous processing:** Redis, BullMQ, and a dedicated Node.js AI worker.
- **AI and retrieval:** LangChain with Google Gemini; Qdrant is provisioned for vector search. The code-question agent currently returns a placeholder response after retrieval, so it is not reported as a completed feature.
- **Infrastructure:** Docker Compose provisions MongoDB, Redis, and Qdrant for local development [Doc26; Mon26; Qdr26].

2.2 Implementation Details

2.2.1 Failure Ingestion and Job Queue

The Express server exposes `/webhooks/github` and verifies GitHub's signature before routing an event. A failed check-run creates a log entry, updates pull-request status, emits a real-time UI event, and enqueues a repair job. The worker consumes `fix-code` and `rag-query` jobs using a Redis connection. This separation limits the effect of slow model calls on the webhook endpoint.

2.2.2 Verification-Gated Fixer

The fixer reads a self-healing prompt, attaches an error log and source files, and requests complete code blocks rather than partial diffs. It supports up to three attempts when parsing or verification fails. Before staging a change, the Git module checks that the configured target is either the bundled test repository or a designated sandbox. It also rejects file paths that resolve outside that target. For each modified JavaScript file with a matching test, the module runs the test locally. A failed test causes the original file content to be restored and prevents a commit. This is the project's central safety control.

When credentials and a remote are configured, a successful verified patch is pushed to a newly created `ai/fix-pr-...` branch and submitted through the GitHub pull-request API [Git26b]. The system does not merge the pull request; a developer must review and merge it.

2.2.3 Dashboard and Observability

The dashboard protects workspace routes behind login and renders a commit/AI node graph, live metrics, error-log viewer, activity feed, and chat drawer. The backend persists run outcome, failing stage, attempt count, branch, pull request URL, and affected files. These records provide a trace of both success and failure, rather than presenting only successful demonstrations.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The repository includes a curated eight-case JavaScript benchmark for exercising the fixer workflow. Cases cover missing imports, off-by-one errors, logic errors, array-method mistakes, cross-file behaviour, misleading comments, incorrect error handling, and stateful behaviour. Each case has a corresponding test file. The evaluation procedure is: (1) run the failing test to obtain the error context, (2) obtain a candidate repair, (3) run the associated test after applying the candidate, and (4) restore the original benchmark file after the run.

The following results are taken from the repository's recorded validation table. They distinguish a test pass from an unqualified reliable repair: the cross-file case passed its test but is marked "gamed" in the supplied record, and the stateful case failed. Consequently, it should not be presented as a general claim that Git-Mind repairs arbitrary multi-file or stateful defects.

3.2 Performance Metrics

Metric	Observed	Interpretation
Benchmark cases	8	Curated JavaScript repair cases in the repository.
Test-passing cases	7/8	Includes one cross-file result marked with a gaming concern.
Unqualified passes	6/8	Passes not flagged as gamed in the recorded validation.
Stateful-case result	0/1	The stateful benchmark did not pass.
Maximum fix attempts	3	Enforced by the fixer-agent retry loop.
Automatic merge	0	Design requires human review of every pull request.

Table 3.1: Repository-recorded validation results

3.3 Analysis

The implementation demonstrates the intended safety boundary: a candidate patch is not committed after a failed local verification, and explicit guards restrict write operations to the test repository or configured sandbox. The project also implements the main integration surfaces—webhook routing, queueing, persistent logs, API endpoints, and dashboard event handling.

The evidence is strongest for the small deterministic benchmark cases. It does not yet establish production reliability, latency, or broad language support. In particular, the placeholder RAG answer, the absence of configured automated package tests in the workspace scripts, and the flagged cross-file result are important limits. Reporting these constraints makes the observed results actionable and avoids confusing an experimental benchmark with a deployment claim.

Chapter 4

Conclusion

4.1 Summary of Work

Git-Mind implements a human-in-the-loop approach to CI-failure remediation. It accepts failure context, uses an LLM to propose code changes, verifies testable changes locally, records outcomes, and can create a separate pull request for a human reviewer. Its modular monorepo architecture connects a React dashboard, Express API, BullMQ worker, MongoDB state, Redis queue, and GitHub integration.

4.2 Key Findings

- Verification before commit is a practical control against untested AI patches and is more defensible than accepting model output directly.
- The benchmark record shows encouraging results for several simple, deterministic JavaScript defects, but one flagged result and one stateful failure show why category-wise reporting is necessary.
- Pull-request-based delivery preserves developer agency: the AI can prepare a fix, while a human retains the merge decision.

4.3 Future Work and Recommendations

1. Complete the RAG answer-generation path with file-level citations and evaluate its answer quality.
2. Extend the benchmark with repeated trials, real CI failures, and separate metrics for single-file, cross-file, and stateful defects.
3. Require the presence of an applicable test before a patch can be committed, rather than proceeding when no matching test is found.
4. Add an explicit filter for AI-authored branches to prevent webhook feedback loops.
5. Measure time-to-pull-request, false-positive rate after review, and queue behaviour under concurrent workloads.

4.4 Final Remarks

The project shows that an AI repair assistant can be useful without being autonomous. Git-Mind's value lies in combining code generation with verification, auditability, scoped permissions, and human review. Further evaluation should expand its capabilities only where evidence supports them.

Bibliography

[Doc26] Docker, Inc. *Docker Compose Documentation*. 2026. URL: https://docs.docker.com/compose/ (cit. on p. 7).

[Git26a] GitHub. *About webhooks*. 2026. URL: https://docs.github.com/en/webhooks/about-webhooks (cit. on p. 5).

[Git26b] GitHub. *REST API endpoints for pull requests*. 2026. URL: https://docs.github.com/en/rest/pulls/pulls (cit. on p. 8).

[Met26] Meta Open Source. *React Documentation*. 2026. URL: https://react.dev/ (cit. on p. 7).

[Mon26] MongoDB, Inc. *MongoDB Documentation*. 2026. URL: https://www.mongodb.com/docs/ (cit. on p. 7).

[Qdr26] Qdrant. *Qdrant Documentation*. 2026. URL: https://qdrant.tech/documentation/ (cit. on p. 7).

[Rea26] React Flow. *React Flow Documentation*. 2026. URL: https://reactflow.dev/ (cit. on p. 7).

[Tas26] Taskforce.sh. *BullMQ Documentation*. 2026. URL: https://docs.bullmq.io/ (cit. on p. 5).